

FORTRAN 90

Arreglos.

El atributo DIMENSION

En Fortran 90 se usa el atributo **DIMENSION** para declarar arreglos. El atributo **DIMENSION** requiere tres componentes para completar la especificación del arreglo *rank*, *shape*, y *extent*.

- El *rank* de un arreglo es el numero de de “índices” o “subíndices.” El máximo rango es 7 (*i.e.*, 7-dimensional).
- El *shape* de un arreglo indica el número de elementos en cada “dimensión.”

El “rank” y “shape” de un arreglo es representado como $(s_1, s_2, \dots, s_n)$, donde n es el rango del arreglo y s_i ($1 < i < n$) es el numero de elementos en la i -ésima dimensión. (7) significa de rango 1 “array” con 7 elementos, (5,9) significa de rango 2 es decir, un arreglo cuya primer y segunda dimensión tienen 5 y 9 elementos, respectivamente. (10,10,10,10) significa de rango 4 “array” que tiene 10 elementos en cada dimensión.

- El *extent* se escribe como $m:n$, donde m y n ($m < n$) son de tipo **INTEGERS**. Hemos visto esto en el **SELECT CASE**, substring, etc.

Cada dimensión tiene su propio extent. Un extent de una dimensión es el rango de su índice. Si m is omitido, el default is 1. $-3:2$ significa que los posibles índices son -3, -2, -1, 0, 1, 2. $5:8$ significa que los posibles índices son 5,6,7,8. 7 significa que los posibles índices son 1,2,3,4,5,6,7.

El atributo **DIMENSION** tiene la siguiente forma:

DIMENSION(extent-1, extent-2, ..., extent- n)

Donde, extent- i es el “extent” de cada dimensión i . Entonces lo anterior significa un arreglo de dimensión n (*i.e.*, n índices) donde la i -ésima dimensión tiene un rango dado por extent- i . **Hay que recordar que:** Fortran 90 solo permite un máximo de 7 dimensiones.

Ejemplos:

- **DIMENSION** (-1:1) es un arreglo 1-dimensional con los índices -1,0,1
- **DIMENSION** (0:2, 3) es un arreglo 2-dimensional con los posibles valores en el primer índice 0,1,2 y el segundo 1,2,3.
- **DIMENSION** (3, 4, 5) es un arreglo 3-dimensional donde los posibles valores del primer índice son 1,2,3, el segundo 1,2,3,4, y el tercero 1,2,3,4,5.

La declaración de un arreglo es muy simple solo hay que agregar el atributo **DIMENSION** al tipo de la variable. Los valores en el atributo **DIMENSION** usualmente son **PARAMETERS** que hacen las modificaciones a un programa faciles.

Ejemplo:

```
INTEGER, PARAMETER :: SIZE=5, LOWER=3, UPPER = 5
INTEGER, PARAMETER :: SMALL = 10, LARGE = 15
REAL, DIMENSION(1:SIZE) :: x
INTEGER, DIMENSION(LOWER:UPPER,SMALL:LARGE) :: a,b
LOGICAL, DIMENSION(2,2) :: Truth_Table
```

Fortran 90 tiene en general, tres diferentes maneras para usar arreglos: refiriéndose a un *elemento del arreglo individual*, refiriéndose a *todo el arreglo*, y refiriéndose a una *sección de un arreglo*. El primero es muy fácil empieza con el nombre del arreglo seguido por `()` entre los cuales están los índices separados por `,`. Hay que notar que cada índice deberá ser un entero o una expresión evaluado a un entero, y el valor del índice deberá estar *en el rango del correspondiente extent*. Sin embargo fortran no checa esto por nosotros.

Supóngase que tenemos las siguientes declaraciones :

```
INTEGER, PARAMETER :: L_BOUND = 3, U_BOUND = 10
INTEGER, DIMENSION(L_BOUND:U_BOUND) :: x
```

```
DO i = L_BOUND, U_BOUND
  x(i) = i
END DO
```

El arreglo `x()` tiene 3,4,5,..., 10

```
DO i = L_BOUND, U_BOUND
  IF (MOD(i,2) == 0) THEN
    x(i) = 0
  ELSE
    x(i) = 1
  END IF
END DO
```

El arreglo tiene `x()` 1 0 1 0 1 0 1 0

Supongamos que tenemos la siguiente declaración:

```
INTEGER, PARAMETER :: L_BOUND = 3, U_BOUND = 10
INTEGER, DIMENSION(L_BOUND:U_BOUND, &
  L_BOUND:U_BOUND) :: a
```

```
DO i = L_BOUND, U_BOUND
  DO j = L_BOUND, U_BOUND
    a(i,j) = 0
  END DO
  a(i,i) = 1
END DO
```

Genera la matriz identidad

```
DO i = L_BOUND, U_BOUND
  DO j = i+1, U_BOUND
    t = a(i,j)
    a(i,j) = a(j,i)
    a(j,i) = t
  END DO
END DO
```

Intercambia la parte inferior de la diagonal por la superior (i.e. la transpuesta)

El DO implícito

Fortran tiene un **DO implícito** que puede generar eficientemente un conjunto de valores y/o elementos. El **DO implícito** es una variación del **DO-loop**. El **DO** tiene la siguiente sintaxis:

`(item-1, item-2, ..., item-n, v=initial, final, step)`

Aquí, `item-1, item-2, ..., item-n` son variables o expresiones, `v` es una variable entera, e `initial, final, y step` son expresiones enteras. “`v=initial, final, step`” es exactamente como se ve un **DO-loop**.

La ejecución de un **DO implícito** a través de una variable `v` que empieza con el valor `initial`, y avanza hasta el valor de `final` con un tamaño de paso `step`. El resultado es una secuencia de números.

- `(i+1, i=1,3)` genera 2, 3, 4.
- `(i*k, i+k*i, i=1,8,2)` genera `k, 1+k (i=1), 3*k, 3+k*3 (i=3), 5*k, 5+k*5 (i=5), 7*k, 7+k*7 (i=7)`.
- `(a(i), a(i+2), a(i*3-1), i*4, i=3,5)` genera `a(3), a(5), a(8), 12 (i=3), a(4), a(6), a(11), 16 (i=4), a(5), a(7), a(14), 20`.

DO implícito puede estar jerarquizado por ejemplo:

`(i*k, (j*j, i*j, j=1,3), i=2,4)`

Arriba tenemos que la expresión: `(j*j, i*j, j=1,3)` esta jerarquizada por el **do implícito** en `i`. Aquí están los resultados:

Cuando `i = 2`, el **DO implícito** genera

`2*k, (j*j, 2*j, j=1,3)`

Entonces, `j` va de 1 a 3 y genera:

`2*k, 1*1, 2*1, 2*2, 2*2, 3*3, 2*3`

`j = 1      j = 2      j = 3`

Cuando `i = 3`, se genera:

`3*k, (j*j, 3*j, j=1,3)`

Entonces el loop para `j` da:

`3*k, 1*1, 3*1, 2*2, 3*2, 3*3, 3*3`

`j = 1      j = 2      j = 3`

Cuando $i = 4$, el loop para i genera:

$4*k, (j*j, 4*j, j=1,3)$

Entonces el loop para j da:

$4*k, 1*1, 4*1, 2*2, 4*2, 3*3, 4*3$

 $j = 1 \quad j = 2 \quad j = 3$

La siguiente expresión genera una tabla de multiplicación:

$((i*j, j=1,9), i=1,9)$

Lo siguiente da como resultado las entradas de una matriz triangular superior *renglón-por-renglón*, de un arreglo de 2 dimensiones:

$((a(p,q), q = p,n), p = 1,n)$

Cuando $p = 1$, el “loop” interno q produce: $a(1,1), a(1,2), \dots, a(1,n)$

Cuando $p = 2$, el “loop” interno q produce: $a(2,2), a(2,3), \dots, a(2,n)$

Cuando $p = 3$, el “loop” interno q produce: $a(3,3), a(3,4), \dots, a(3,n)$

Cuando $p = n$, el “loop” interno q produce: $a(n,n)$

Lo siguiente produce las entradas de una matriz triangular superior *columna-por-columna*:

$((a(p,q), p = 1,q), q = 1,n)$

Cuando $q = 1$, el “loop” interno p produce: $a(1,1)$

Cuando $q = 2$, el “loop” interno p produce: $a(1,2), a(2,2)$

Cuando $q = 3$, el “loop” interno p produce: $a(1,3), a(2,3), a(3,3)$

Cuando $q = n$, el “loop” interno p produce: $a(1,n), a(2,n), a(3,n), \dots, a(n,n)$

En el DO implícito puede usarse las instrucciones `READ(*,*)` y `WRITE(*,*)`.

Cuando un DO implícito es usado éste es equivalente a ejecutar las instrucciones de I/O como si se generarán elementos. Por ejemplo, lo siguiente genera e imprime una tabla de multiplicar

`WRITE(*,*) ((i, "*", j, "=", i*j, j=1,9), i=1,9)`

Lo siguiente tiene un mejor formato (*i.e.*, 9 renglones):

```
DO i = 1, 9
WRITE(*,*) (i, "*", j, "=", i*j, j=1,9)
END DO
```

Lo siguiente muestra tres maneras de leer n datos y ponerlos en arreglo unidimensional $a()$:

(1) `READ(*,*) n, (a(i), i=1, n)`

(2) `READ(*,*) n`
`READ(*,*) (a(i), i=1, n)`

(3) `READ(*,*) n`
`DO i = 1, n`
`READ(*,*) a(i)`
`END DO`

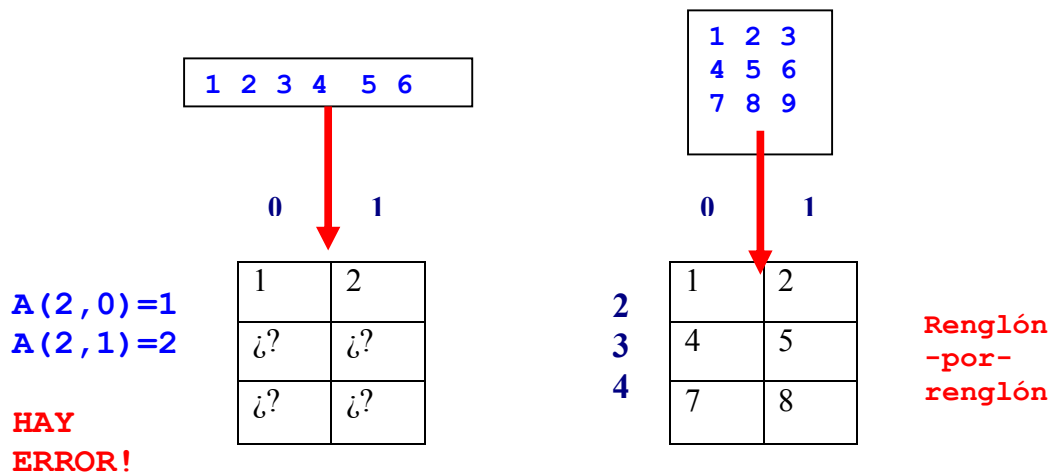
Supongamos ahora que tenemos un arreglo bidimension $a()$:

`INTEGER, DIMENSION(2:4,0:1) :: a`

Supongamos además que tenemos una instrucción `READ` de la siguiente manera:

`READ(*,*) ((a(i,j), j=0,1), i=2,4)`

¿Cuales son los resultados para las siguientes entradas?



Supóngase ahora que tenemos un arreglo bidimensional `a()`:

```
INTEGER, DIMENSION(2:4,0:1) :: a
DO j = 0, 1
  READ(*,*) (a(i,j), i=2,4)
END DO
```

¿Cuales son los resultados para las siguientes entradas?

`A(2,0)=1`
`A(3,0)=2`
`A(4,0)=3`

**HAY
ERROR!**

1	2	3	4	5	6
---	---	---	---	---	---

0 1

1	¿?
2	¿?
3	¿?

1	2	3
4	5	6
7	8	9

0 1

2
3
4

1	4
2	5
3	6

Columna
-por-
columna

Ejemplo: la multiplicación de matrices

Se lee una matriz de $l \cdot m$, $A_{l \cdot m}$ y una matriz de $m \cdot n$, $B_{m \cdot n}$, y calcula su producto $C_{l \cdot n} = A_{l \cdot m} \times B_{m \cdot n}$.

```
PROGRAM Matrix_Multiplication
IMPLICIT NONE
INTEGER, PARAMETER :: SIZE = 100
INTEGER, DIMENSION(1:SIZE,1:SIZE) :: A, B, C
INTEGER :: L, M, N, i, j, k
```

```
READ(*,*) L, M, N ! read sizes <= 100
```

```
DO i = 1, L
  READ(*,*) (A(i,j), j=1,M) ! A() is L-by-M
END DO
```

```
DO i = 1, M
  READ(*,*) (B(i,j), j=1,N) ! B() is M-by-N
END DO
```

```
DO i = 1, L
  DO j = 1, N
    C(i,j) = 0 ! for each C(i,j)
    DO k = 1, M ! (row i of A)*(col j of B)
      C(i,j) = C(i,j) + A(i,k)*B(k,j)
    END DO
  END DO
END DO
```

```
DO i = 1, L ! print row-by-row
  WRITE(*,*) (C(i,j), j=1, N)
```

```
END PROGRAM Matrix_Multiplication
```

Arrays as Arguments:

En Fortran 90 cuando pasamos un arreglo a una función o subrutina automáticamente se pasa el *shape* al argumento formal. Un subprograma recibe la “shape” y usa la cota inferior para cada “extent” y así recuperar la cota superior.

```

INTEGER, DIMENSION(2:10) :: Store
.....
CALL Funny(Score)

```

shape is (9)

```

SUBROUTINE Funny(x)
IMPLICIT NONE
INTEGER, DIMENSION(-1:), INTENT(IN) :: x
..... other statements .....
END SUBROUTINE Funny

```

(-1:7)

Un ejemplo más:

```

REAL, DIMENSION(1:3,1:4) :: x
INTEGER :: p = 3, q = 2
CALL Fast(x,p,q)

```

shape is (9)

```

SUBROUTINE Fast(a,m,n)
IMPLICIT NONE
INTEGER, INTENT(IN) :: m,n
REAL, DIMENSION(-m:,n:), INTENT(IN) :: a
..... other statements .....
END SUBROUTINE Fast

```

(-m:,n:) pasa a (-3:-1,2:5)

El atributo **ALLOCATABLE**

En muchas situaciones, se conoce exactamente el tamaño de un arreglo es posible entonces declarar un arreglo sin decir su tamaño. Es aquí donde el atributo **ALLOCATABLE** funciona, este atributo lo que indica es que en el momento de la declaración lo único que se conoce es el rango del arreglo a declarar pero no u tamaño. Así, en lugar del tamaño se tienen solamente los dos puntos **::**.

```

INTEGER, ALLOCATABLE, DIMENSION(:) :: a
REAL, ALLOCATABLE, DIMENSION(:, :) :: b
LOGICAL, ALLOCATABLE, DIMENSION(:, :, :) :: c

```


La instrucción `ALLOCATE`

La instrucción `ALLOCATE` tiene la siguiente sintaxis:

```
ALLOCATE (array-1,...,array-n,STAT=v)
```

Donde, `array-1`, ..., `array-n` son los nombres de los arreglos con sus tamaños igual que en el atributo `DIMENSION` y `v` es una variable entera. Después de la ejecución de `ALLOCATE`, Si `v` ≠ 0, entonces al menos uno de los arreglos no obtuvo la memoria deseada.

Ejemplo:

```
REAL,ALLOCATABLE,DIMENSION(:) :: a
LOGICAL,ALLOCATABLE,DIMENSION(:,:) :: x
INTEGER :: status
ALLOCATE(a(3:5), x(-10:10,1:8), STAT=status)
```

`ALLOCATE` solo reserva espacio para arreglos con el atributo `ALLOCATABLE`. Los tamaños en `ALLOCATE` pueden usar expresiones `enteras`. Hay que estar seguros que las variables han sido inicializadas debidamente. Ejemplo:

```
INTEGER,ALLOCATABLE,DIMENSION(:,:) :: x
INTEGER,ALLOCATABLE,DIMENSION(:) :: a
INTEGER :: m, n, p
READ(*,*) m, n
ALLOCATE(x(1:m,m+n:m*n),a(-(m*n):m*n),STAT=p)
IF (p /= 0) THEN
    ..... reportar error aquí .....

```

Si `m = 3` y `n = 5`, entonces tendremos

```
x(1:3,8:15) y a(-15:15)
```

La instrucción `DEALLOCATE`

Para desocupar la memoria utilizada por arreglos es posible utilizar la instrucción `DEALLOCATE()` que mostramos a continuación:

```
DEALLOCATE (array-1,...,array-n,STAT=v)
```

Donde, `array-1`, ..., `array-n` son los nombres de los arreglos que ocupan espacio, y `v` es una variable entera. Si la “deallocation” falla (e.g., algunos arreglos fueron “deallocated”), el valor en `v` no es cero. Después de la “deallocation” de un arreglo, este no es accesible y cualquier intento de acceder marcará un error en el programa.

```
DEALLOCATE(a b c STAT=status)
```

La función intrínseca `ALLOCATED`

La función `ALLOCATED(a)` regresa `.TRUE.`.
Si el arreglo dinámico o “`ALLOCATABLE`” ha sido correctamente asignado.
De otra manera se regresa `.FALSE.`

```
INTEGER, ALLOCATABLE, DIMENSION(:) :: Mat
INTEGER :: status
ALLOCATE(Mat(1:100), STAT=status)
..... ALLOCATED(Mat) returns .TRUE. ....
..... other statements .....
DEALLOCATE(Mat, STAT=status)
..... ALLOCATED(Mat) returns .FALSE. ....
```